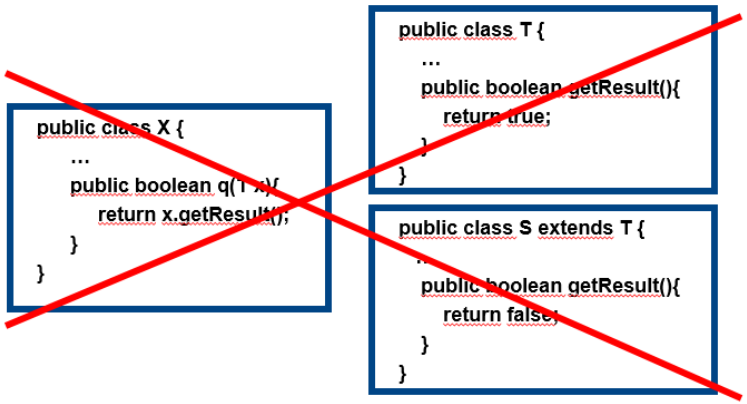
Hochschule Bochum
TECHNIK WIRTSCHAFT GESUNDHEIT

Objektorientierte Programmiertechniken



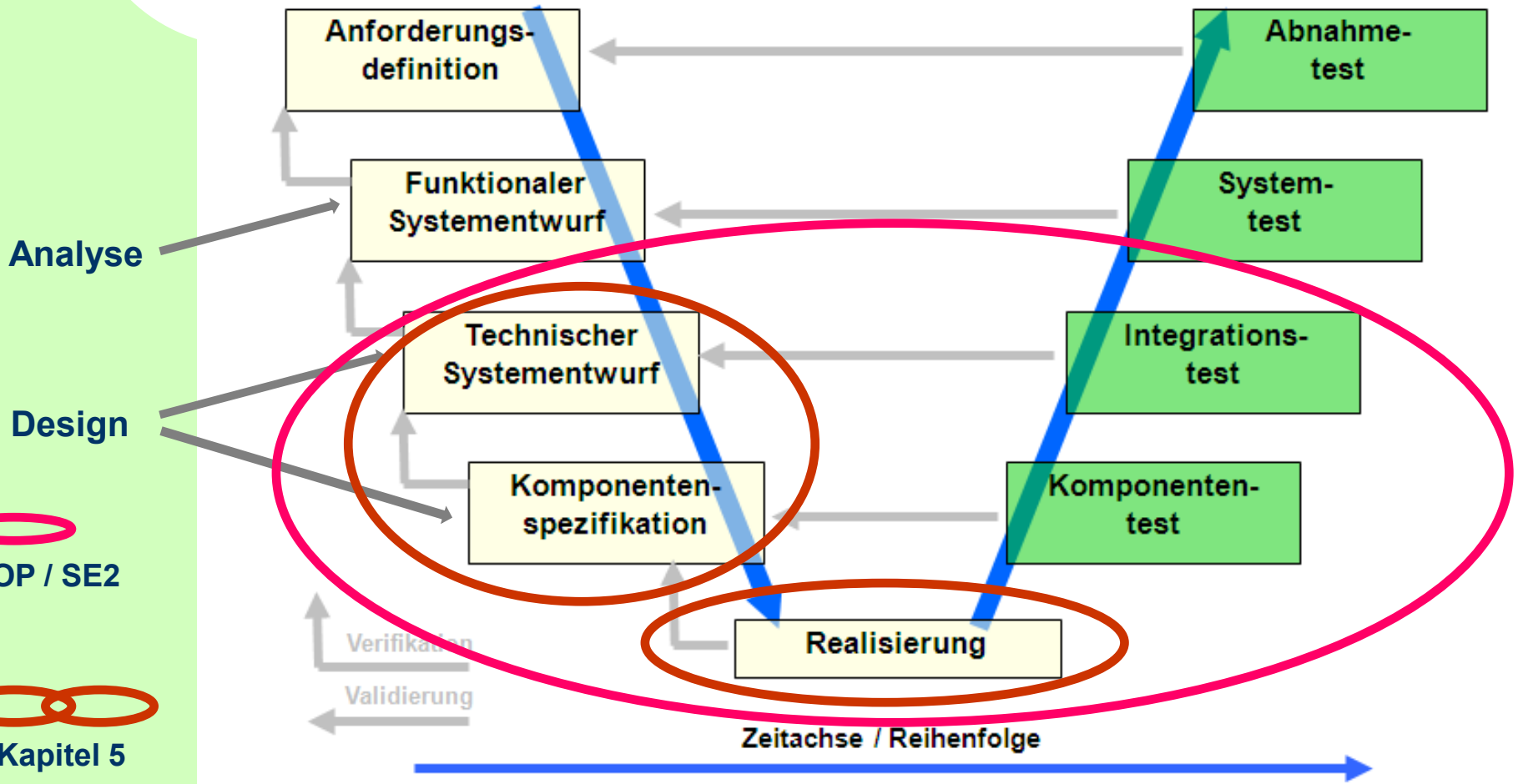
Prof. Dr. Ursula Oesing

SOLID / Clean Code

- Vorspann
- SOLID
- Clean Code
- Fazit

Vorspann

- **Einordnung der Veranstaltung**



Vorspann

- **Wieso SOLID?**
  - SOLID-Prinzipien führen zu einer höheren Wartbarkeit und Lebensdauer von Software sowie zu einer Reduktion der Fehleranfälligkeit.
- **Wieso Clean Code?**
  - Clean Code führt zu einer höheren Wartbarkeit und Lebensdauer von Software sowie zu einer Reduktion der Fehleranfälligkeit.

Vorspann

SOLID

Clean Code

Fazit

SOLID

Vorspann

SOLID

Clean Code

Fazit

Was sind SOLID-Prinzipien?

- eine Gruppe von fundamentalen Prinzipien für gutes objektorientiertes Design.
- SOLID steht für:
  - **S**ingle Responsibility Principle
  - **O**pen Closed Principle
  - **L**iskov Substitution Principle
  - **I**nterface Segregation Principle
  - **D**ependency Inversion Principle

Eigenschaften

- in Textform formuliert
- programmiersprachenunabhängig, nicht fertig implementiert.

SOLID

Vorspann

SOLID

Clean Code

Fazit

Single Responsibility Principle

(Prinzip der einzelnen Verantwortlichkeit)

- Jede Klasse sollte eine strikt abgegrenzte Verantwortlichkeit besitzen. Vermeiden Sie es, Klassen mehr als eine Aufgabe zu geben. Beispiel:
- **Robert C. Martin:**
  (Agile Software Development: Principles, Patterns, and Practices)
  A class should have only one reason to change.
- Erweiterung: Übertragung auf Methoden

Beispiele für solche Verantwortlichkeiten

nach Craig Larman

- etwas wissen; Daten oder Infos über ein Konzept kennen
- etwas können; Steuerungs- oder Kontrollverantwortung
- etwas erzeugen

SOLID

- **Single Responsibility Principle**
- **Fragen**
  - Was bedeutet die Einhaltung dieses Prinzips für Model-Klassen?
  - Was bedeutet die Einhaltung dieses Prinzips für Entitätsklassen?
  - Ergänzung: Was bedeutet die Einhaltung dieses Prinzips für Methoden?

Vorspann

SOLID

Clean Code

Fazit

SOLID

- **Single Responsibility Principle**
- **Fragen**
  - Inwiefern wird in dem folgenden Beispiel gegen das Single Responsibility Principle verstoßen? Abhilfe?

```
package personen;
public class Person {

    private String vorname;
    private String nachname;
    private String strasseHNr;
    private int plz;
    private String ort;

    // Konstruktor und get- und set-Methoden
    …
}
```

Vorspann

SOLID

Clean Code

Fazit

SOLID

- **Single Responsibility Principle**
- **Fragen**
  - Inwiefern wird in dem folgenden Beispiel gegen das Single Responsibility Principle verstoßen? Abhilfe?

```
package personen;
public class PersonModel {

    private String vorname;
    private String nachname;

    // Konstruktor und get- und set-Methoden

    public void schreibePersonInDatei(){
        …
    }
    …
}
```

Vorspann

SOLID

Clean Code

Fazit

SOLID

Vorspann

SOLID

Clean Code

Fazit

Wieso Single Responsibility Principle?

- Mehr als eine Verantwortung für eine Klasse führt zu mehreren Bereichen, in denen zukünftige Änderungen notwendig werden können. Die Wahrscheinlichkeit, dass die Klasse zu einem späteren Zeitpunkt geändert werden muss, steigt zusammen mit dem Risiko, sich bei solchen Änderungen subtile Fehler einzuhandeln.
- Dieses Prinzip führt in der Regel zu Klassen mit hoher Kohäsion, in denen alle Methoden einen starken gemeinsamen Bezug haben.
- enthalten im orangenen Grad von Clean Code, siehe unten

SOLID

Vorspann

SOLID

Clean Code

Fazit

Open Closed Principle

(Prinzip der Offen- und Verschlossenheit)

- Ein Modul, eine Klasse, eine Methode sollen offen für Erweiterungen sein, aber geschlossen gegen Modifikationen.
- **Bertrand Meyer** (Object Oriented Software Construction): *Modules should be both open (for extension) and closed (for modification).*

Ausbau eines Moduls eines Softwareprodukts um Funktionalität

- durch Hinzufügen einer Klasse (ggf. Vererbung) / Methode (**Erweiterung**) anstatt durch Änderung einer Klasse / Methode (**Modifikation**).
- Änderung einer Klasse / Methode nur zur Verbesserung der Qualität oder zur Korrektur.

SOLID

- Open Closed Principle
- Aufgabe

Vorspann

SOLID

Clean Code

Fazit

```
public class Vereinsmitglied {

        private int mitgliedsnummer;
        private String vorname;
        private String nachname;
        private char mitgliedsart;
        public static final char PASSIVES_VEREINSMITGLIED = 'p';
        public static final char AKTIVES_VEREINSMITGLIED = 'a';

        // Konstruktor, get- und set- Methoden

        public int berechneJahresbeitrag() {
                int ergebnis = 0;
                if(this.mitgliedsart == PASSIVES_VEREINSMITGLIED) {
                        ergebnis = 120;
                }
                else if(this.mitgliedsart == AKTIVES_VEREINSMITGLIED) {
                        ergebnis = 180;
                }
                return ergebnis;
        }
}
```

SOLID

- **Open Closed Principle**
- **Fortsetzung der Aufgabe**

Erweitern Sie das Programm. Organisierende Vereinsmitglieder sollen 20 Euro Ermäßigung bekommen. Beachten Sie dabei das Open Closed Principle!

```
import java.util.ArrayList;

public class Main {

	public static void main(String[] args) {
		ArrayList<Vereinsmitglied> mitglieder = new ArrayList<>();
		mitglieder.add(
			new Vereinsmitglied(1, "Erika", "Musterfrau", 'a'));
		mitglieder.add(
			new Vereinsmitglied(2, "Max", "Mustermann", 'p'));
		System.out.println(
			"Beiträge der Vereinsmitglieder pro Jahr:");
		for(Vereinsmitglied vm : mitglieder) {
			System.out.println(vm.berechneJahresbeitrag()
				+ " Euro für " + vm.getVorname() + " "
				+ vm.getNachname());
		}
		mitglieder.clear();
	}
}
```

```
Problems  Javadoc  Declaration  Console  Debug
<terminated> Main [Java Application] C:\Program Files\Java\jdk-11
Beiträge der Vereinsmitglieder pro Jahr:
180 Euro für Erika Musterfrau
120 Euro für Max Mustermann
```


Vorspann

SOLID

Clean Code

Fazit

SOLID

Wieso Open Closed Principle?

- Die vorhandenen Module, Klassen, Methoden werden bei einer Fortentwicklung der Software nicht geändert. Dadurch wird vermieden, dass in dem bestehenden Programm Fehler einprogrammiert werden.
- vgl.: https://clean-code-developer.de/die-grade/gruener-grad/#Open_Closed_Principle

  Warum?

  Weil das Risiko, durch neue Features ein bisher fehlerfreies System zu instabilisieren, so gering wie möglich gehalten werden sollte.
- enthalten im grünen Grad von Clean Code, siehe unten

Vorspann

SOLID

Clean Code

Fazit

SOLID

Vorspann

SOLID

Clean Code

Fazit

Liskov Substitution Principle

(Liskov' sches Ersetzbarkeitsprinzip)

- Unterklassen (abgeleitete Klassen) sollen anstelle ihrer Oberklassen einsetzbar sein.
- Eine Instanz einer Unterklasse muss sich so verhalten, dass jemand, der meint, ein Objekt der Oberklasse vor sich zu haben, nicht durch unerwartetes Verhalten überrascht wird, wenn es sich dabei tatsächlich um ein Objekt der Unterklasse handelt.

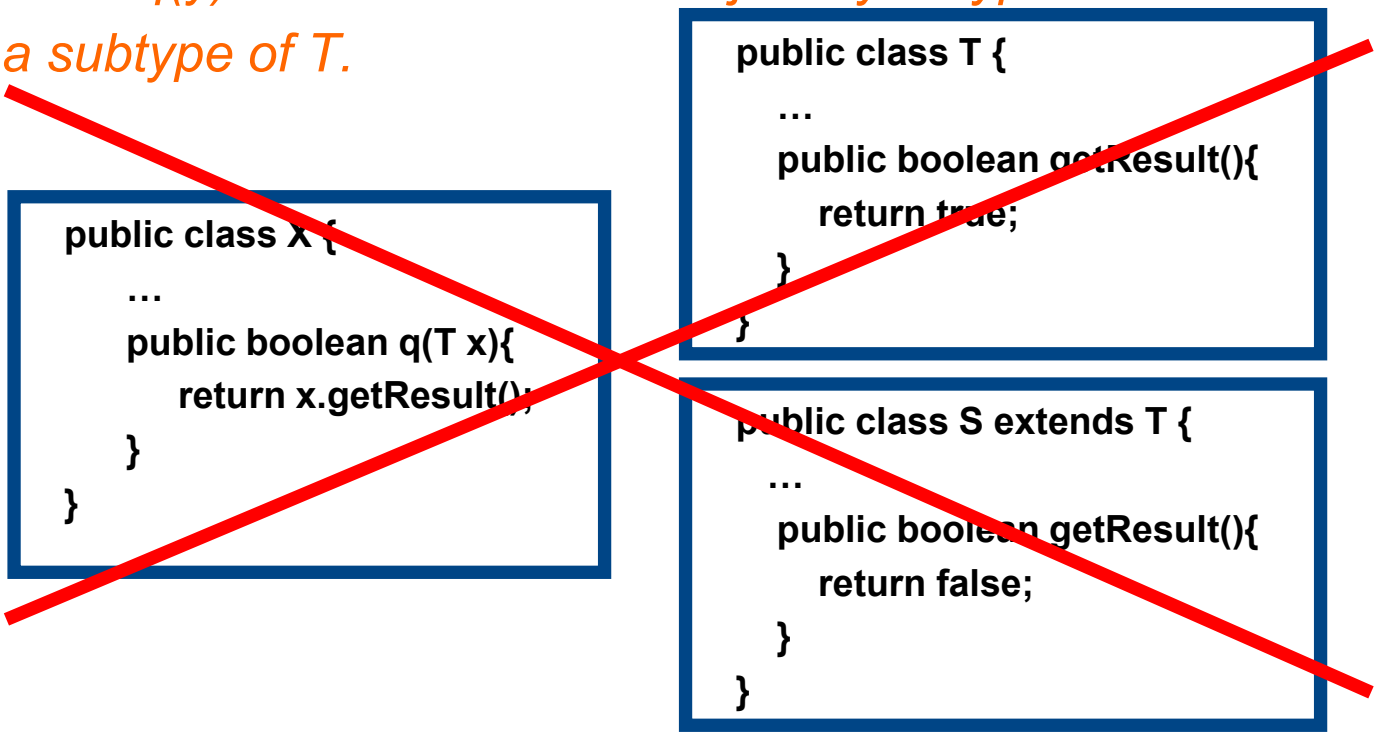
Beispiele

- Sie sollten in Unterklassen niemals Methoden der Oberklassen durch leere Implementierungen überschreiben.
- Oberklassen und auch Interfaces dürfen keine Methoden enthalten, die nicht für alle Mitglieder der Klassenhierarchie passen.

SOLID

➢ Liskov Substitution Principle

- **Barbara Liskov und Jeannette Wing:**
  (Behavioral Subtyping using Invariants and Constraints)
  Let q(x) be a property provable about objects x of type T. Then q(y) should be true for objects y of type S where S is a subtype of T.



Vorspann
SOLID
Clean Code
Fazit

SOLID

➢ Liskov Substitution Principle

- **Beispiel:**

```
import javax.swing.JOptionPane;

public class X {
    public static void main(String[ ] args){
        try{
            Integer.parseInt(JOptionPane.showInputDialog(
                "Bitte geben Sie Ihr Geburtsjahr ein!"));
        }
        catch(Exception exc){
            Meldung x = new Meldung();
            x.gibMeldungAus("Das Format des Geburtsjahrs ist nicht korrekt.");
          // Meldung x = new Fehlermeldung();
          // x.gibMeldungAus("Das Format des Geburtsjahrs ist nicht korrekt.");
        }
    }
}
```

Was sind q(x) bzw. q(y) und T und S?

Vorspann

SOLID

Clean Code

Fazit

SOLID

Liskov Substitution Principle

- **Fortsetzung des Beispiels:**

```
import javax.swing.JOptionPane;

public class Meldung{

    public void gibMeldungAus(String meldung){
        JOptionPane.showMessageDialog(
            null, meldung);
    }
}
```

Abhilfe?

```
public class Fehlermeldung extends Meldung{

    @Override
    public void gibMeldungAus(String meldung){
        System.out.println(meldung);
    }
}
```

Vorspann

SOLID

Clean Code

Fazit

SOLID

- **Liskov Substitution Principle**
- **Frage**
  - Inwiefern wird in dem folgenden Beispiel gegen das Liskov Substitution Principle verstoßen? Abhilfe?

Vorspann

SOLID

Clean Code

Fazit


```
public class Schraubendreher {

    private long identnummer;
    private float preis;
    private String klingenform;

    // Konstruktor und
    // get- und set-Methoden
    …
}
```

```
public class Schleifgeraet
    extends Schraubendreher{

    private int leistung;
    private int gewicht;
    private int leerlaufDz;

    public String getKlingenform(){
        return "Es gibt keine Klingenform.";
    }

    // Konstruktor und weitere
    // get- und set-Methoden
    …
}
```

SOLID

Vorspann

SOLID

Clean Code

Fazit

Wieso Liskov Substitution Principle?

- Durch das Prinzip wird garantiert, dass Operationen vom Typ *Oberklasse*, die auf ein Objekt des Typs *Unterklasse* angewendet werden, auch korrekt ausgeführt werden.
- Ein Objekt vom Typ *Unterklasse* kann dann bedenkenlos durch ein Objekt vom Typ *Oberklasse* ersetzt werden.

Notiz

- enthalten im gelben Grad von Clean Code, siehe unten

SOLID

Vorspann

SOLID

Clean Code

Fazit

Interface Segregation Principle (Schnittstellenaufteilungsprinzip)

- Große Schnittstellen sollen in mehrere kleinere Schnittstellen aufgeteilt werden, falls implementierende Klassen ansonsten unnötige Methoden haben müssen.
- **Robert C. Martin:**
  Clients should not be forced to depend upon interfaces that they do not use.

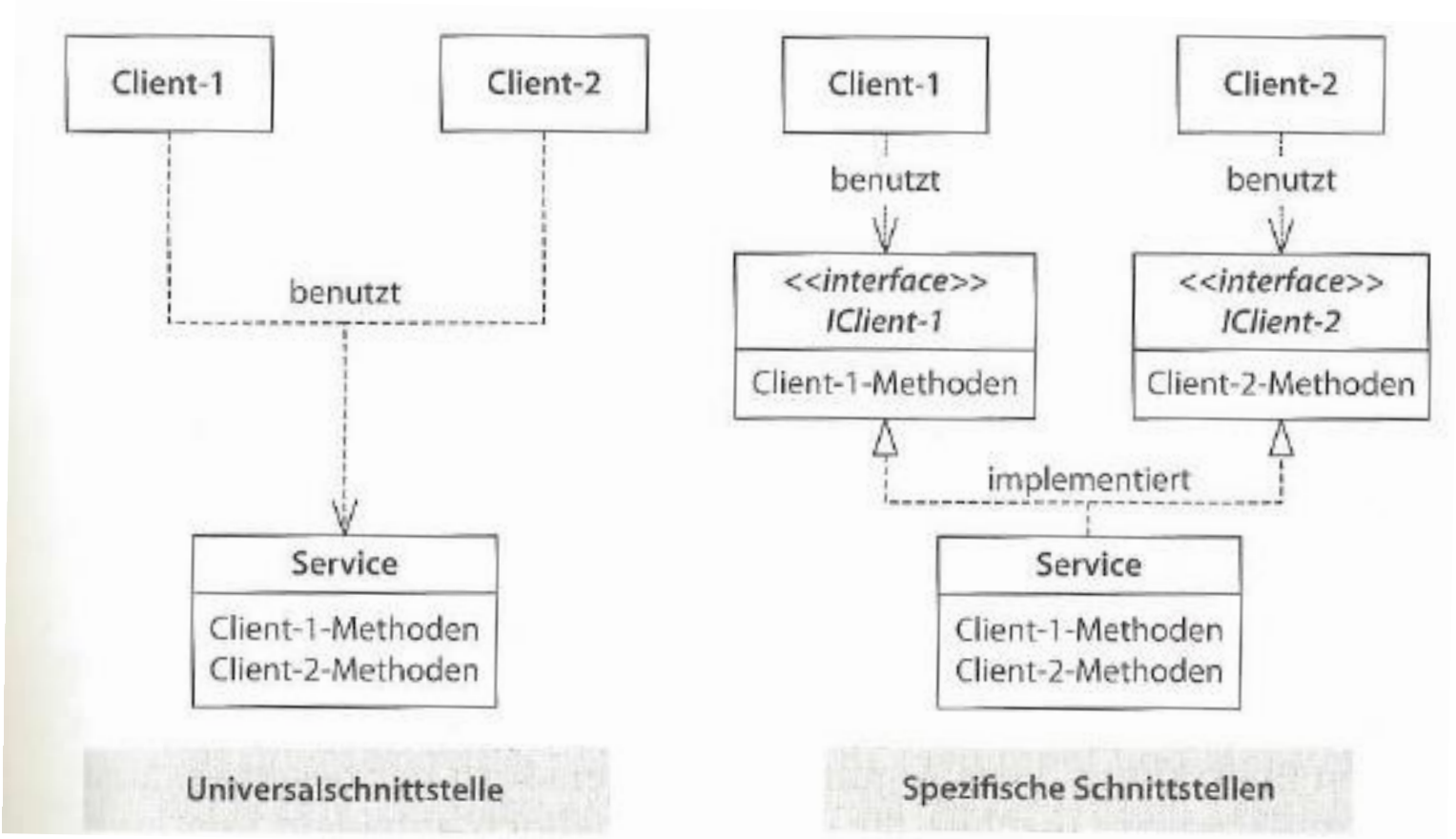
Anwendung des Interface Segregation Principles

- Aufteilen eines Interfaces in zwei (oder mehr) geeignete Interfaces.
- Verwenden eines geeigneten Interfaces, falls bereits vorhanden.

SOLID

➢ Interface Segregation Principle

Abbildung aus Karl Eilebrecht, Gernot Starke: Patterns kompakt



Vorspann

SOLID

Clean Code

Fazit

SOLID

➢ Interface Segregation Principle

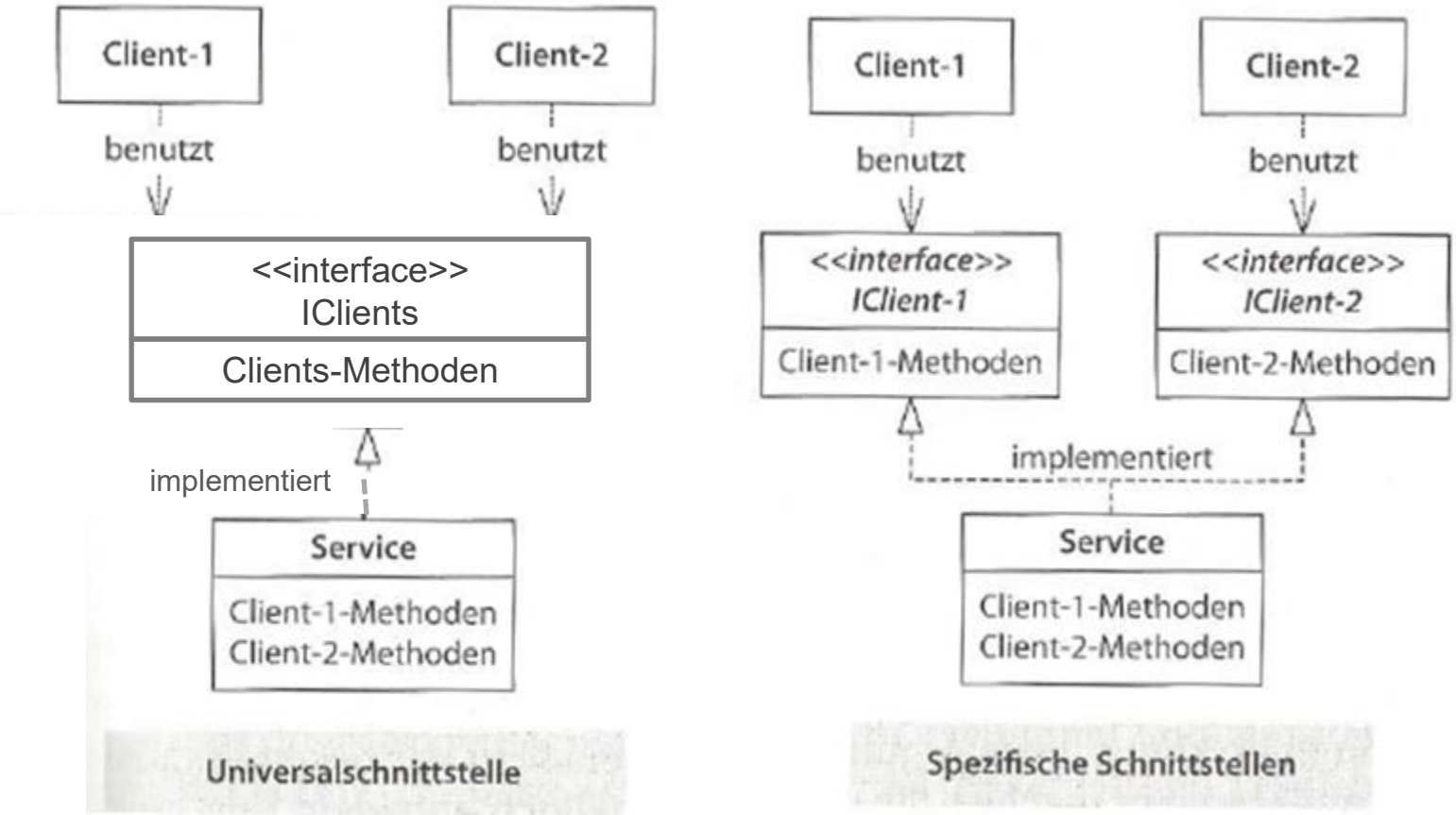
vgl. Abbildung aus Karl Eilebrecht, Gernot Starke: Patterns kompakt

Vorspann

SOLID

Clean Code

Fazit



SOLID

- **Interface Segregation Principle**
- **Frage**
  - Inwiefern wird in dem folgenden Beispiel gegen das Interface Segregation Principle verstoßen? Abhilfe?

Vorspann

SOLID

Clean Code

Fazit

```
public class PersonendatenBeschaffung
      implements ImportExport {
      …

      public void schreibeInCsvDatei(){}
      public void schreibeInTxtDatei(){}
      public void leseAusCsvDatei(){
          ...
      }

      public void leseAusTxtDatei(){
          ...
      }
}
```

```
public interface ImportExport{

      public void schreibeInCsvDatei();
      public void schreibeInTxtDatei();
      public void leseAusCsvDatei();
      public void leseAusTxtDatei();
}
```

SOLID

- **Interface Segregation Principle**
- **Fragen**

  siehe Beispiel vom OpenClosed-Principle

  Geänderte Klasse Main:

  In wiefern wird das Interface Segregation Principle nicht eingehalten?

  Abänderung?

Vorspann

SOLID

Clean Code

Fazit


```
import java.util.*;

public class Main {

    private static void gibBeitraegeVereinsmitgliederAus(
        List<Vereinsmitglied> mitglieder) {
        System.out.println("Beiträge der Vereinsmitglieder pro Jahr:");
        for (Vereinsmitglied vm : mitglieder)        {
            System.out.println(vm.berechneJahresbeitrag()
              + " Euro für " + vm.getVorname() + " "
              + vm.getNachname());
        }
    }

    public static void main(String[] args) {
        List<Vereinsmitglied> mitglieder = new ArrayList<>();
        mitglieder.add(new Vereinsmitglied(1, "Erika", "Musterfrau", 'a'));
        mitglieder.add(new Vereinsmitglied(2, "Max", "Mustermann", 'p'));
        gibBeitraegeVereinsmitgliederAus(mitglieder);
        mitglieder.clear();
    }
}
```

```
Problems  Javadoc  Declaration  Console  Debug
<terminated> Main [Java Application] C:\Program Files\Java\jdk-11
Beiträge der Vereinsmitglieder pro Jahr:
180 Euro für Erika Musterfrau
120 Euro für Max Mustermann
```

SOLID

Vorspann

SOLID

Clean Code

Fazit

Wieso Interface Segregation Principle ?

- Mit Hilfe des Interface-Segregation Principles ist es möglich, eine Software derart in entkoppelte und somit leichter refaktorisierbare Klassen aufzuteilen, dass zukünftige fachliche oder technische Anforderungen an die Software nur geringe Änderungen an der Software selbst benötigen.
- enthalten im gelben Grad von Clean Code, siehe unten
- vgl.: https://clean-code-developer.de/die-grade/gelber-grad/#Interface_Segregation_Principle_ISP

Warum?

Segregation bedeutet Abtrennung. Das Prinzip besagt, dass ein Client nicht von Details eines Service abhängig sein soll, die er gar nicht benötigt. Je weniger in dessen Interface enthalten ist, desto geringer ist die Kopplung zwischen den beiden Komponenten.

SOLID

Wieso Interface Segregation Principle ?

- vgl.: https://clean-code-developer.de/die-grade/gelber-grad/#Interface_Segregation_Principle_ISP

Warum?

Stellen wir uns vor, wir müssten einen Stecker planen, mit dem ein Monitor an einen Computer angeschlossen werden soll. Wir entscheiden uns, einfach alle Signale die in einem Computer so anfallen, per Stecker zur Verfügung zu stellen. Der hat dann zwar einige Hundert Pins, aber dafür ist er maximal flexibel. Dummerweise ist damit die Kopplung ebenfalls maximal.

Beim Beispiel des Steckers ist es offensichtlich, dass eine Monitor-verbindung nur jene Signale enthalten soll, die zur Darstellung eines Bildes auf dem Monitor erforderlich sind. Genauso verhält es sich mit Software Interfaces. Auch sie sollten so klein wie möglich sein, um unnötige Kopplung zu vermeiden. Und genau wie beim Monitorstecker sollte das Interface eine hohe Kohäsion haben: Es sollte nur Dinge enthalten, die wirklich eng zusammen gehören.

SOLID

Vorspann

SOLID

Clean Code

Fazit

Dependency Inversion Principle

(Abhängigkeits-Umkehr-Prinzip)

- Das Dependency Inversion Principle besagt, dass Module höherer Ebenen nicht von konkreten Implementierungen aus Modulen niedriger Ebenen abhängen sollen.
- **Robert C. Martin:**

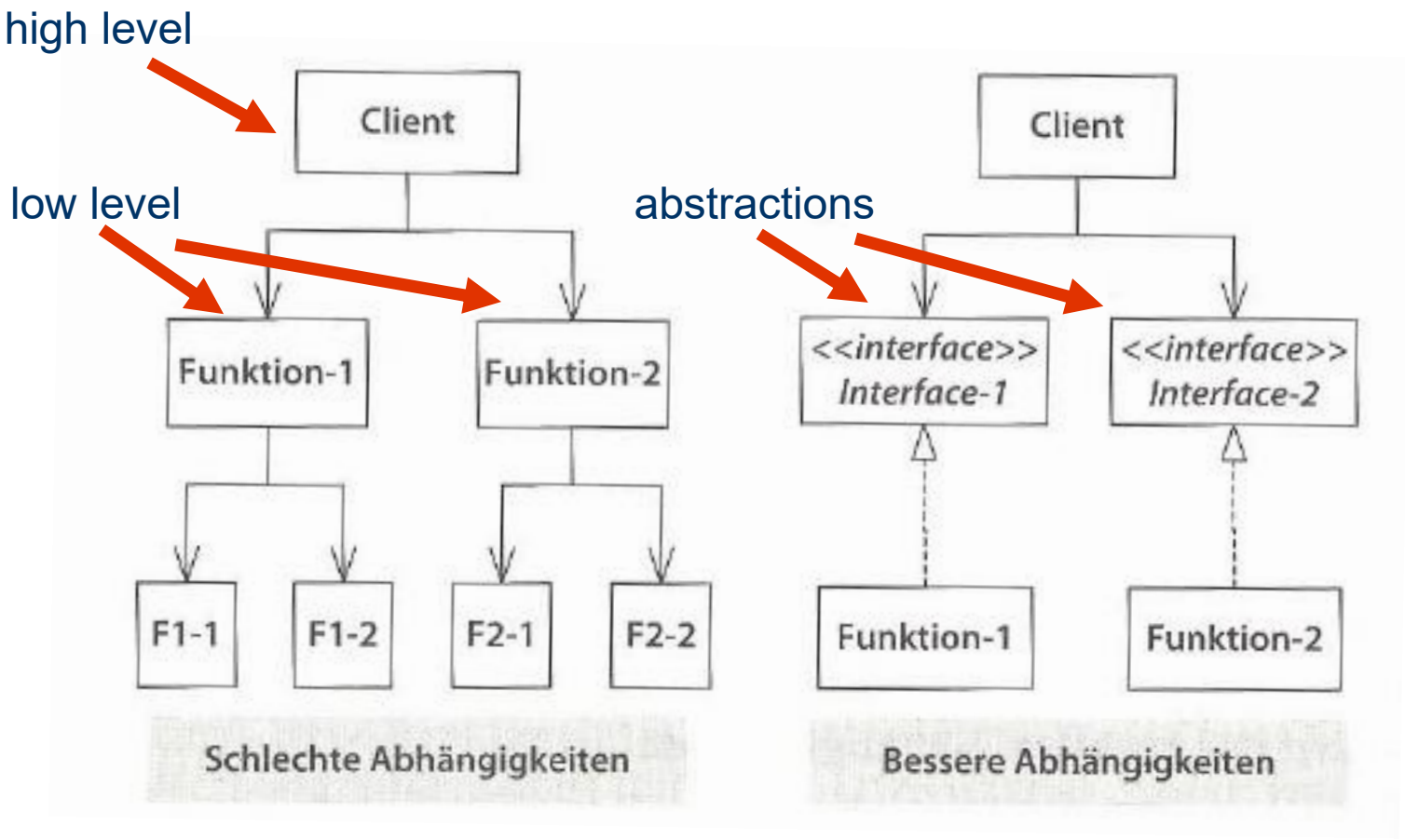
  A. High level modules should not depend on low level modules. Both should depend on abstractions.

  B. Abstractions should not depend upon details. Details should depend upon abstractions.

SOLID

➢ Dependency Inversion Principle

Abbildung aus Karl Eilebrecht, Gernot Starke: Patterns kompakt



Vorspann

SOLID

Clean Code

Fazit

SOLID

- **Dependency Inversion Principle**
- **Frage**
  - Gegeben ist das folgende Beispiel. Inwiefern wird das Dependency Inversion Principle nicht eingehalten? Abhilfe?

```
<terminated> Anwendersystem (1) [J
Tagesmietpreise:
62
```


Vorspann

SOLID

Clean Code

Fazit


```
public class Auto {

    private String kennzeichen;
    private String marke;
    private int ps;

    // Konstruktor, get- und set-Methoden …

    public int berechneTagesmietpreis() {
        return getPs() / 2;
    }
}
```

```
import java.util.*;
public class Anwendungssystem {

    public static void gibTagesmietpreiseAus(
        Iterable<Auto> vermietbareObjekte) {
        System.out.println("Tagesmietpreise:");
        for(Auto v : vermietbareObjekte) {
            System.out.println(v.berechneTagesmietpreis());
        }
    }

    public static void main(String[] args) {
        List<Auto> vermietbareObjs = new ArrayList<>();
        vermietbareObjs.add(
            new Auto("BO YQ 435", "Peugeot 305", 125));
        gibTagesmietpreiseAus(vermietbareObjs);
    }
```

SOLID

Dependency Inversion Principle

Fortsetzung: Aufgabe

- Gegeben ist ist eine Klasse Werkzeug. Ein Werkzeug ist ebenfalls vermietbar. Erweitern Sie die Anwendung
  a) ohne Verwendung des Dependency Inversion Principle und
  b) mit Verwendung des Dependency Inversion Principle.

  Ein Werkzeug soll in die Liste vermietbareObjs aufgenommen werden. Der Tagesmietpreis soll ebenfalls ausgegeben werden.

```
<terminated> Anwendersystem (3
Tagesmietpreise:
62
1
```

```
public class Werkzeug {

    private int identnummer;
    private int basispreis;
    private String werkzeugart;

    // Konstruktor, get- und set-Methoden …

    public int berechneKaufpreis(){
        return (int) Math.round(basispreis * 1.19;
    }
    public int berechneTagesmietpreis() {
        return berechneKaufpreis() / 15;
    }
}
```

Vorspann
SOLID
Clean Code
Fazit

SOLID

Wieso Dependency Inversion Principle?

- Änderungen von Modulen niedriger Ebenen sollen nicht zu unerwünschte Seiteneffekten bei Modulen höherer Ebenen führen. Module niedriger Ebene sollen einfach austauschbar sein.
- enthalten im gelben Grad von Clean Code, siehe unten
- vgl.: https://clean-code-developer.de/die-grade/gelber-grad/#Dependency_Inversion_Principle

Warum?

Verwendet eine High-Level Klasse eine Low-Level Klasse unmittelbar, so ergibt sich eine starke Kopplung zwischen beiden. Spätestens beim Versuch, die High-Level Klasse isoliert zu testen, wird man auf Schwierigkeiten stoßen. Aus diesem Grund sollte die High-Level Klasse von einem Interface abhängig sein, das wiederum von der Low-Level Klasse implementiert wird. So kann die Low-Level Klasse im Unit Test durch ein Mockup ersetzt werden.

Clean Code

Vorspann

SOLID

Clean Code

Fazit

Clean Code

vgl. https://clean-code-developer.de

Eine Initiative für mehr Professionalität in der Softwareentwicklung

- Kernfrage: Was macht eigentlich den professionellen Softwareentwickler / die professionelle Softwareentwicklerin aus?
- Leitsatz: *Professionalität = Bewusstheit + Prinzipien*
- Gründung der Initiative CCD: Clean Code Developer von Stefan Lieser und Ralf Westphal
- Es gibt
  - ein Wertesystem
  - Tugenden
  - eine Einteilung in Grade

Clean Code

Vorspann

SOLID

Clean Code

Fazit

Wertesystem

- Beispiele zu einem **Wertesystem**:
  Ein professioneller Softwareentwickler hat ein inneres Wertesystem, das aus Folgendem besteht:
  - Wandelbarkeit: Die Kosten für das Einbauen eines neuen Features in ein bestehendes Softwaresystem sollen nicht von der Anzahl der bestehenden Features abhängen.
    (ehemalig Evolvierbarkeit)
  - Korrektheit: Der Code soll frei von Fehlern sein.
  - Produktionseffizienz: Das Herstellen von Software soll so kostengünstig wie möglich sein.
  - Kontinuierliche Verbesserung: Durch Reflexion sollen sich nicht nur der Entwickler / die Entwicklerin, sondern auch das Team, das Softwareunternehmen und schließlich die gesamte Branche weiter entwickeln.

Clean Code

- Vorspann
- SOLID
- **Clean Code**
- Fazit

Die Tugenden

Inhaltsverzeichnis

Prinzipielles

1. Schätze Variation (Value Variation (VV))
2. Tue nur das Nötigste (Do Only What´s Neccessary (DOWN))
3. Isoliere Aspekte (Isolate Aspects (IA))
4. Minimiere Abhängigkeiten (Minimize Dependencies (MD))
5. Halte Versprechen ein (Honor Pledges (HP)

Praktisches

1. Umarme Unsicherheit (Embrace Uncertainty (EU))
2. Fokussiere (Focus (F))
3. Wertschätze Qualität (Value Quality (VQ))
4. Mach fertig (Get Things Done (GTD))
5. Halte Ordnung (Stay Clean (SC))
6. Bleib am Ball (Keep Moving (KM))

https://clean-code-developer.de/die-tugenden/

Clean Code

Vorspann

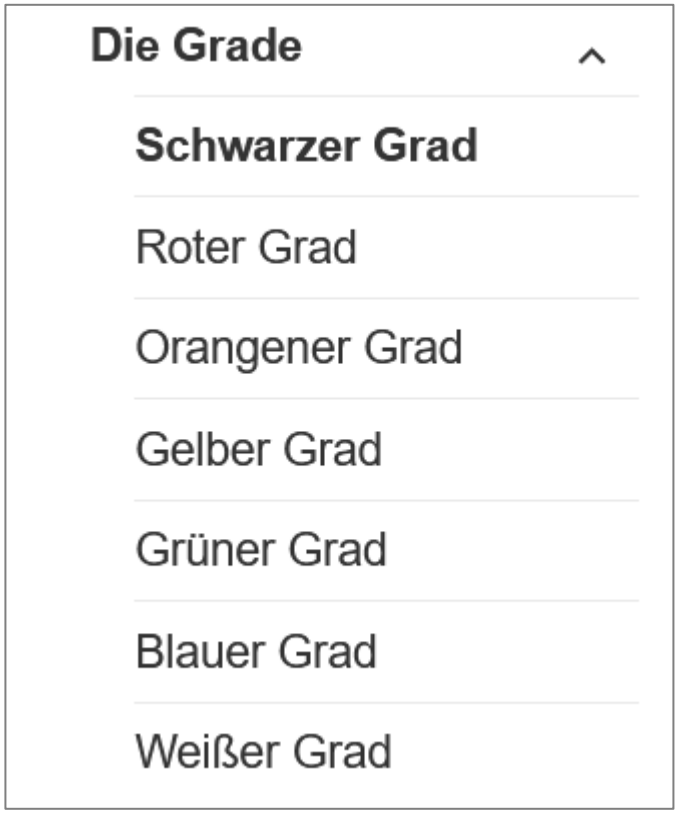
SOLID

Clean Code

Fazit

Grade des Clean Code Developments

- Unterschiedliche Stufen des Clean Code Developments sind mit unterschiedlichen Farben markiert.
- **schwarz**
  motivierte und an Clean Code interessierte Entwickler*innen, die, aus welchen Gründen auch immer, noch nicht damit beginnen können



Clean Code

Grade des Clean Code Developments



Clean Code

- **Ergänzung zum Integration Operation Segregation Principle**
  - Beispiel zu Operation

Vorspann

SOLID

Clean Code

Fazit

```
public double berechnePreisJeArtikel(int anzahlKaufArtikel)
    throws IllegalArgumentException {
    double ergebnisProz = 100;
    int[ ] anzArtikel = new int[]{0, 500, 1000};
    int[ ] preisJeStueckProz = new int[ ]{100, 75, 60};
    int pos = 1;
    if(anzahlKaufArtikel >= 1000){
        ergebnisProz = 60;
    }
    else{
        while(anzArtikel[pos] < anzahlKaufArtikel) {
            pos++;
        }
        ergebnisProz = ((preisJeStueckProz[pos]  - preisJeStueckProz[pos-1]) * 1.0
            / (anzArtikel[pos] - anzArtikel[pos-1]))
            * (anzahlKaufArtikel - anzArtikel[pos-1]) +  preisJeStueckProz[pos-1];
    }
    return (ergebnisProz / 100) * this.basispreis;
}
```

Clean Code

Negativbeispiel zum IOSP

Vorspann

SOLID

Clean Code

Fazit

```
public double berechnePreisJeArtikel(int anzahlKaufArtikel) {
    try{
        double ergebnisProz = 100;
        int[ ] anzArtikel = new int[]{0, 500, 1000};
        int[ ] preisJeStueckProz = new int[ ]{100, 75, 60};
        int pos = 1;
        …
        return (ergebnisProz / 100) * this.basispreis;
    }
    catch(IllegalArgumentExcepiton iaExc){
        zeigeFehlermeldungAn(
            "Falsche Vorgabe der Anzahl der Artikel");
    }
    return -1;
}
```

Clean Code

- **Refaktorisierungsmuster zur Durchführung von DRY**
- **Aufgabe zum Pattern Extract Method**
  - Gegeben seien zwei Methoden zum Schreiben von Daten in Dateien (ohne das Pattern Factory Method). Lagern Sie gemeinsame Codeteile in eine weitere Methode aus.

```
public void schreibeGeburtsdatumInCsvDatei()
      throws IOException{
      BufferedWriter aus = new BufferedWriter(
            new FileWriter("Geburtsdaten.csv"));
      aus.write(geburtsdatum.getGeburtstag() + ";"
            + geburtsdatum.getGeburtsmonat() + ";"
            + geburtsdatum.getGeburtsjahr());
      aus.close();
}

public void schreibeGeburtsdatumInTxtDatei()
      throws IOException{
      BufferedWriter aus = new BufferedWriter(
            new FileWriter("Geburtsdaten.txt"));
      aus.write(geburtsdatum.getGeburtstag() + "."
            + geburtsdatum.getGeburtsmonat() + "."
            + geburtsdatum.getGeburtsjahr());
      aus.close();
}
```

Clean Code

Vorspann

SOLID

Clean Code

Fazit

- **Refaktorisierungs-muster zur Durch-führung von DRY**
- **Aufgabe zum Pattern Chain Constructor**
  - Gegeben sei die bisherige Klasse Geburtsdatum. Gegeben sei weiterhin eine Klasse Person. Wenden Sie das Pattern Chain Constructor an.

```
public class Person {

    private String vorname;
    private String nachname;
    private Geburtsdatum geburtsdatum;

    public Person(String vorname, String  nachname) {
        this.vorname = vorname;
        this.nachname = nachname;
    }

    public Person(String vorname, String nachname,
        Geburtsdatum geburtsdatum) {
        this.vorname = vorname;
        this.nachname = nachname;
        this.geburtsdatum = geburtsdatum;
    }
    // get- und set-Methoden
    ...
}
```

Clean Code

Grade des Clean Code Developments

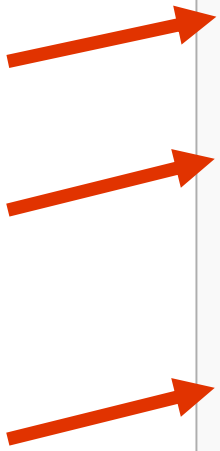
- **orange**

Vorspann

SOLID

Clean Code

Fazit



Inhaltsverzeichnis

2. Grad der Clean Code Developer

Prinzipien

- Single Level of Abstraction (SLA)
- Single Responsibility Principle (SRP)
- Separation of Concerns (SoC)
- Source Code Konventionen

Praktiken

- Issue Tracking
- Automatisierte Integrationstests
- Lesen, Lesen, Lesen
- Reviews

Clean Code

Grade des Clean Code Developments

- **gelb**

Vorspann

SOLID

Clean Code

Fazit



Fazit

Wieso SOLID?

- SOLID-Prinzipien führen zu einer höheren Wartbarkeit und Lebensdauer von Software sowie zu einer Reduktion der Fehleranfälligkeit.

Vorspann

SOLID

Clean Code

Fazit

Fazit

Vorspann

SOLID

Clean Code

Fazit

Wieso Clean Code?

- Clean Code führt zu einer höheren Wartbarkeit und Lebensdauer von Software sowie zu einer Reduktion der Fehleranfälligkeit.
- Inwiefern tragen die Prinzipien und Praktiken des **roten Grades** zum Erreichen der oben genannte Ziele bei?

Inhaltsverzeichnis

1. Grad der Clean Code Developer Initiative

Prinzipien

- Don´t Repeat Yourself (DRY)
- Keep it simple, stupid (KISS)
- Vorsicht vor Optimierungen!
- Favour Composition over Inheritance (FCoI)
- Integration Operation Segregation Principle (IOSP)

Praktiken

- Die Pfadfinderregel beachten
- Root Cause Analysis
- Ein Versionskontrollsystem einsetzen
- Einfache Refaktorisierungsmuster anwenden
- Täglich reflektieren

Fazit

- **Welchen Einfluss hat Clean Code auf das Projektmanagement?**

Vorspann

SOLID

Clean Code

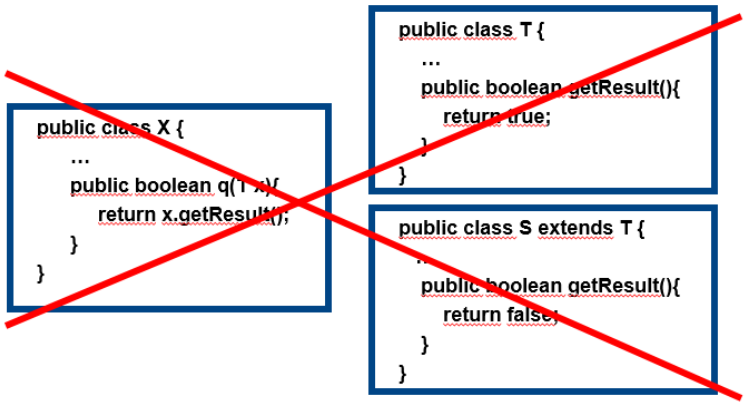
Fazit

SOLID / Clean Code

- Vorspann ✓
- SOLID ✓
- Clean Code ✓
- Fazit ✓

Hochschule Bochum
TECHNIK WIRTSCHAFT GESUNDHEIT

Objektorientierte Programmiertechniken



Vielen Dank für Ihr Interesse!